

Programare orientată pe obiecte

Motor de căutare -documentație-

Stingheriu Alexandru
gr. 3123b

1. Introducere și scopul proiectului

Acest proiect reprezintă un motor de căutare local, conceput pentru a indexa și interoga volume mari de fișiere text (testat pe volume de peste 5 GB). Obiectivul principal a fost crearea unei aplicații de consolă ultra-rapide, care să găsească orice cuvânt în timp constant, folosind concepte avansate de Programare Orientată pe Obiecte (POO) în C++.

În loc să folosesc baze de date externe, am construit „de la zero” structura de date și algoritmi de indexare, punând un accent masiv pe optimizarea memoriei RAM și a vitezei de citire de pe disc.

2. Concepte arhitecturale și structuri de date

Pentru a obține o viteză de căutare instantanee, aplicația se bazează pe două concepte fundamentale:

2.1. Inverted Index (Index Inversat)

În loc ca programul să scaneze fișierele de fiecare dată când utilizatorul caută ceva, el face munca grea o singură dată. Când un folder este adăugat, aplicația mapează fiecare cuvânt găsit la documentul în care se află și reține frecvența acestuia. Rezultatul este un dicționar global unde cheia este cuvântul, iar valoarea este lista de documente.

2.2. Hash Table (Tabelă de Dispersie)

Pentru a stoca acest Inverted Index în memorie, am folosit un Hash Table (`std::unordered_map` din C++). Această structură garantează o complexitate a căutării de $O(1)$. Practic, timpul de căutare este instantaneu, indiferent dacă indexul conține zece cuvinte sau zece milioane.

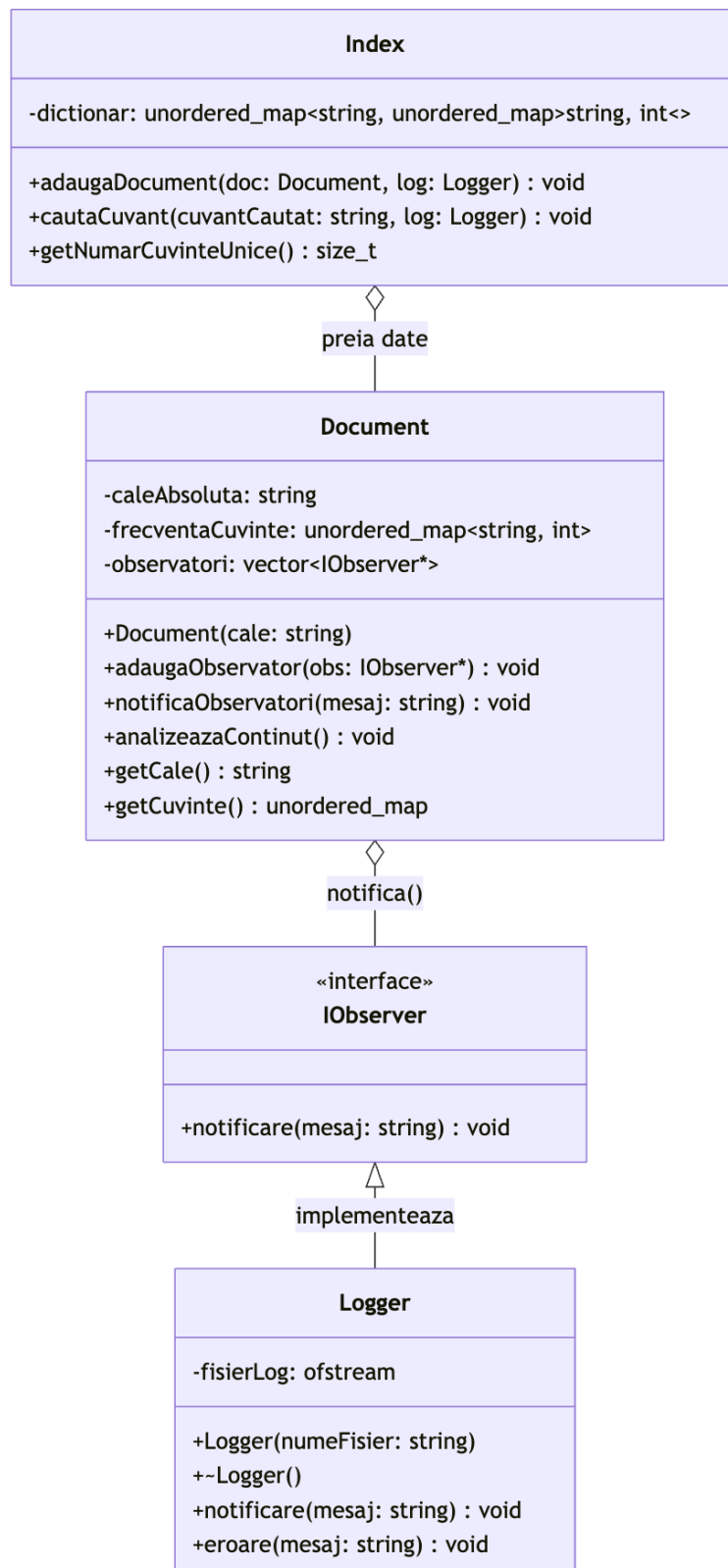
```
C Index.h  X
src > C Index.h > ...
1  #pragma once
2  #include <string>
3  #include <unordered_map>
4  #include "Document.h"
5  #include "Logger.h"
6
7  class Index {
8  private:
9      // Structura optimizată pentru index inversat:
10     // Cuvânt -> {Cale Document -> Număr Apariții}
11     std::unordered_map<std::string, std::unordered_map<std::string, int>> dictionar;
12
13 public:
14     // Preia datele dintr-un document analizat și le integrează în dicționarul general
15     void adaugaDocument(const Document& doc, Logger& log);
16
17     // Caută cuvântul instantaneu (O(1)) și afișează rezultatele
18     void cautaCuvant(const std::string& cuvântCăutat, Logger& log);
19     size_t getNumarCuvinteUnice() const;
20 };
```

2.3. Polimorfism: Modelul Observer

Aplicația ține un istoric al acțiunilor (când se indexează ceva sau când se face o căutare). Pentru a decupla logica de căutare de logica de scriere în fișier, am implementat Design Pattern-ul Observer. Clasa Logger moștenește interfața IObserver și este notificată de clasele principale ori de câte ori are loc un eveniment important.

2.4. Arhitectura Claselor (Diagrama UML)

Pentru a vizualiza relațiile de moștenire (Observer) și compoziție dintre clasele principale ale motorului de căutare, am realizat următoarea diagramă UML:



3. Funcționalități și interacțiunea cu utilizatorul

Interacțiunea cu motorul de căutare se face printr-un meniu CLI (Command Line Interface) intuitiv, protejat împotriva erorilor utilizatorului (de exemplu, nu poți căuta ceva dacă nu ai indexat un folder în prealabil).

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

aki@Akis-MacBook-Air Stingheriu_Alexandru_3123B_MotorCautare-main % ./motor_cautare
[INFO] === APLICATIE PORNITA ===

=====
                MOTOR DE CAUTARE - MENIU PRINCIPAL
=====
[1] Indexeaza un director
[2] Cauta un cuvant
[3] Afiseaza statistici index
[4] Iesire din aplicatie
=====
Alege o optiune (1-4): █
```

Filtru NLP (Stop Words) In-Memory

Nu toate cuvintele aduc valoare unei căutări. Pentru a menține indexul "curat", am implementat un filtru care rulează în timpul parsării:

- Orice cuvânt sub 3 litere este ignorat.
- Cuvintele de legătură (ex: "este", "pentru", "daca", "care") sunt verificate împotriva unei liste `std::unordered_set` și ignorate.

Dacă utilizatorul încearcă să caute un astfel de cuvânt, programul nu dă o eroare seacă, ci îi explică educativ de ce cuvântul a fost ignorat de sistem.

```
Alege o optiune (1-4): 2

--- Mod Cautare (scrie '!menu' pentru a reveni la meniu) ---
Cauta: pentru

[INFO] 'pentru' este un cuvant de legatura (Stop Word).
        Sistemul l-a ignorat pentru a optimiza memoria RAM.
=====
Cauta: la

[INFO] Cuvantul 'la' este prea scurt. A fost filtrat la indexare.
=====
Cauta: masina
[INFO] Utilizatorul a cautat cuvantul: masina

>>> Cuvantul 'masina' a fost gasit in 2 document(e):
-> ./teste_stop_words/test_scurte.txt (de 1 ori)
-> ./teste_stop_words/test_mixt.txt (de 1 ori)
=====
Cauta: █
```

4. Optimizări Extreme (Viteză și Memorie)

Cea mai mare provocare a proiectului a fost citirea rapidă a fișierelor foarte mari (zeci/sute de megabytes) fără a bloca sistemul.

Funcția clasică `std::getline` din C++ nu făcea față fișierelor fără prea multe "Enter-uri" (Newline-uri). Așa că am rescris complet modul de citire a fișierelor folosind Buffered I/O (Citire Binară în Blocuri).

În clasa `Document`, fișierul este citit direct de pe disc în blocuri fixe de exact 1 Megabyte o dată. Calculatorul procesează acel megaoctet direct în memoria RAM, apoi trece la următorul. Acest design izolează consumul de memorie și previne blocajele hardware, permițând motorului să mănânce gigabytes întregi de text cu un consum minim de resurse.

```
const size_t DIMENSIUNE_BUFFER = 1048576; // 1 MB
std::vector<char> buffer(DIMENSIUNE_BUFFER);

std::string cuvantCurent;
cuvantCurent.reserve(128);

while (fisier.read(buffer.data(), DIMENSIUNE_BUFFER) || fisier.gcount() > 0) {
    std::streamsize bytesCititi = fisier.gcount();

    for (std::streamsize i = 0; i < bytesCititi; ++i) {
        char c = buffer[i];
```

5. Mediul de Dezvoltare și Portabilitate

Pentru ca aplicația să fie cât mai robustă, dezvoltarea inițială s-a realizat într-un mediu nativ de Linux, utilizând WSL (Windows Subsystem for Linux) cu distribuția Ubuntu.

Această alegere m-a ajutat din mai multe motive:

- A permis testarea codului direct pe uneltele standard POSIX (compilatorul GNU GCC/G++).
- A facilitat folosirea utilitarului `make` pentru automatizarea compilării. În loc să depind de setările specifice ale unui anumit IDE (cum ar fi Visual Studio), am scris un `Makefile` curat.
- Gestionarea fișierelor cu `std::filesystem` a fost testată riguros în ierarhia de directoare Linux.

Datorită acestui mod independent de a scrie codul, proiectul este acum 100% cross-platform. Poate fi compilat și rulat instantaneu atât pe Ubuntu (sau WSL), cât și pe macOS, folosind exact aceleași comenzi în consolă (`make` urmat de `./motor_cautare`). Nu necesită nicio librărie externă instalată.

6. Concluzii

Acest motor de căutare demonstrează cum o arhitectură corectă (Inverted Index) combinată cu structurile de date potrivite (Hash Tables) poate rezolva probleme de performanță masive. Codul este scalabil, respectă regulile de încapsulare și poate fi oricând extins (de exemplu, adăugând interfețe grafice noi) fără a modifica logica de bază a creierului aplicației.